

Testing Workshop

ICCS Summer School 2025

Dominic Orchard and Isaac Akanho



UNIVERSITY OF
CAMBRIDGE



Institute of
Computing for
Climate Science

Schmidt Sciences

Validation

Did we implement the right equations?

VS

Verification

Did we implement the equations right?

Challenge

Telling these two apart when results are not as expected

Terminology: what does “*verified*” mean?

Verification wrt. a specification

i.e. `check(implementation, specification)`

∴ validation is verification

where specification $\triangleq \approx_{\text{observation}}$

*The value of a specification is what we make of it;
it depends on our goals and values*

Validation as a proxy for verification

- Can we get a stable run (over decades)?
- Is it plausible from physics perspective?
- Do hindcasts reproduce observational record?

If the science is relatively settled, then points to bugs not invalidity

Problem: error localisation is poor!

Our focus: Testing

- Implementation language = specification language
- Test subset of inputs / program behaviours
 - Does not show “absence of bugs” (Dijkstra); may fail to expose unconceived-of bugs
- Key principles:
 - automate testing so that it is easy to deploy
 - speed up development time by reducing bug hunting time
 - increase confidence in code
 - enable safe(r) refactoring (*want semantics preservation*)
- We will use `python` and `pytest`

Plan

- Part 0 - Smoke tests
- Part 1 - Unit tests
- Part 2 - Integration tests
- Part 3 - Property-based tests
- Live demos throughout + exercises based on a 0D Energy Balance model
<https://github.com/Cambridge-ICCS/testing-workshop>

```
pip install -r requirements.txt
```

(or with pip3)

Schedule

Session 1 - 1h

- 10 minute intro about correctness and testing
- 50 minutes explaining concepts about unit testing including
 - Parameterised tests
 - Fixtures
 - Negative tests
 - Approximation and floating point
 - TDD
 - Code coverage

Session 2 - 1h30

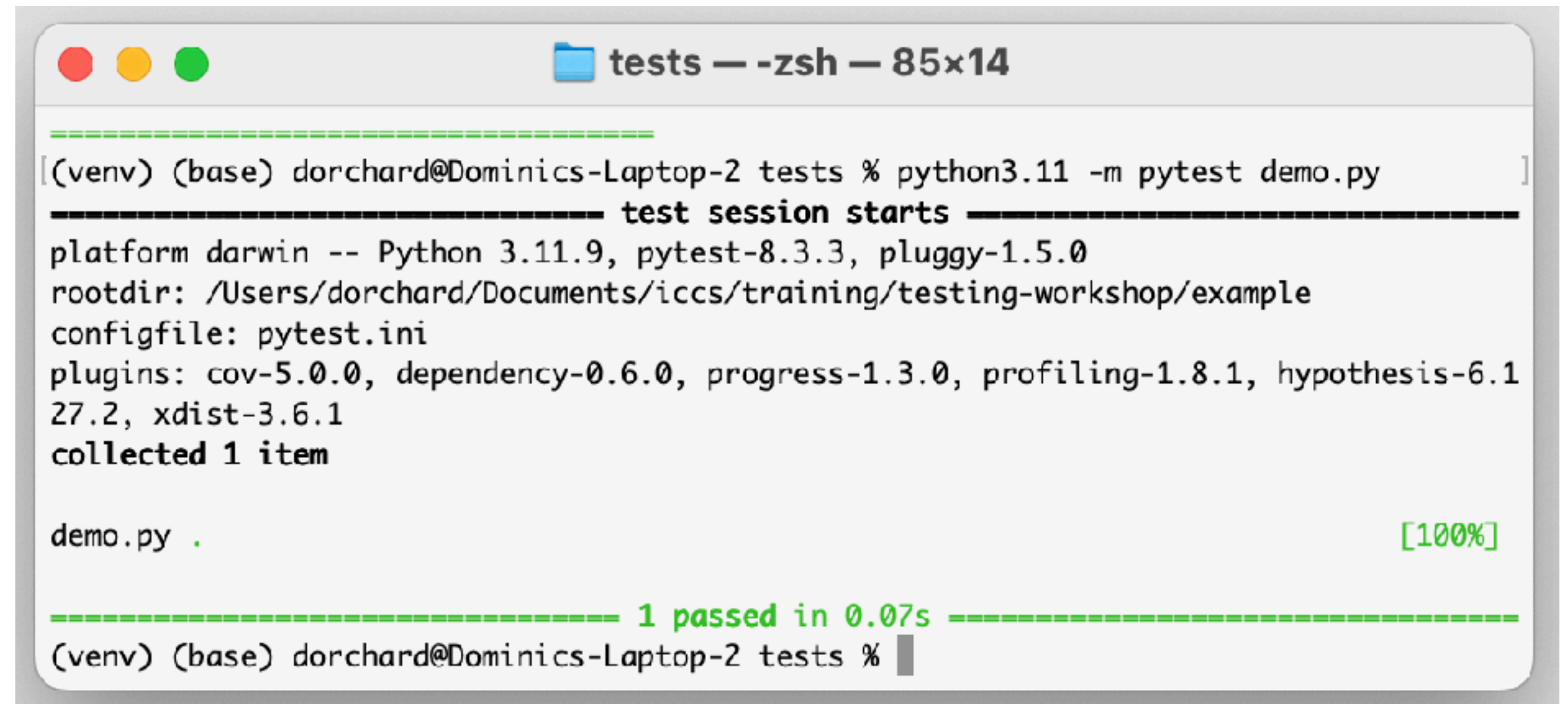
- 20 minute unit test exercises
- 40 minutes lecture
 - Integration and end-to-end tests
 - Property-based testing
- 30 minutes exercises (property-based test exercises)

pytest basics

- Put tests as .py files in a `tests/` directory
- Each test group is a function with prefix `test_`
- Use `assert` keyword with a boolean expression

tests/demo.py

```
def func(x):  
    return x + 1  
  
def test_answer():  
    assert func(3) == 4
```



```
tests — -zsh — 85x14  
-----  
[ (venv) (base) dorchard@Dominics-Laptop-2 tests % python3.11 -m pytest demo.py ]  
----- test session starts -----  
platform darwin -- Python 3.11.9, pytest-8.3.3, pluggy-1.5.0  
rootdir: /Users/dorchard/Documents/iccs/training/testing-workshop/example  
configfile: pytest.ini  
plugins: cov-5.0.0, dependency-0.6.0, progress-1.3.0, profiling-1.8.1, hypothesis-6.1  
27.2, xdist-3.6.1  
collected 1 item  
  
demo.py . [100%]  
  
----- 1 passed in 0.07s -----  
(venv) (base) dorchard@Dominics-Laptop-2 tests %
```


assert statement anatomy

```
assert boolean-expr
```

```
assert boolean-expr, string-expr
```

If boolean-expr false, show string-expr in the output

Part 0 - Smoke tests

What is a smoke test?

🔥 *Did it catch fire?*

- Was the program able to “start” (initialise) without throwing an exception?
 - *Might include loading configuration files or data sets*

```
def test_config_loading():  
    # Check that init does not fail  
    energy_balance.init()  
    # and that it initializes a global variable  
    assert energy_balance.solar_constant is not None, "Solar constant should be initialized"
```


Part I - Unit tests

Unit test basics (1)

- Applied to smaller “units” of code...
- Run code, then compare **actual** results against **expected** results

square : float → float

square(2) ≡ 4

Unit test basics (2)

- Try various values *systematically*:

- with simple / base cases

$\text{square}(0) \equiv 0$

$\text{square}(1) \equiv 1$

- with more complex cases

$\text{square}(4) \equiv 16$

$\text{square}(-3) \equiv 9$

- with corner cases (*requires more creative thought sometimes*)

$\text{square}(\text{NaN}) ???$

Floating point considerations...

- $\text{NaN} \neq \text{NaN}$ use `math.isnan` instead
- Floating-point rounding error can be a pain, `np.isclose` / `np.allclose`
- `pytest.approx`
 - e.g., `pytest.approx(0.614618, abs=1e-4)`
 - ▶ `rel` - Relative tolerance (good when similar scale)
 - ▶ `abs` - Absolute tolerance (good when different scales)

Unit test demo

tests/test_helpers.py

```
from helpers import square
from math import nan, isnan

def test_square():
    assert square(0) == 0
    assert square(1) == 1
    assert square(4) == 16
    assert square(-1) == 1
    assert isnan(square(nan))
```

Test *parameterization*

Reduce repetition by defining a single test over different inputs

tests/test_helpers.py

```
from helpers import square
from math import nan, isnan

def test_square():
    assert square(0) == 0
    assert square(1) == 1
    assert square(4) == 16
    assert square(-1) == 1
    assert isnan(square(nan))
```



tests/test_helpers_param.py

```
from helpers import square
from math import nan, isnan
import pytest

def test_square_nan():
    assert isnan(square(nan))

@pytest.mark.parametrize("value, expected", [
    (0, 0),
    (1, 1),
    (4, 16),
    (-1, 1),
])
def test_square_parametrized(value, expected):
    assert square(value) == expected
```

Test fixtures

Provide context or content for tests

tests/testing.py

```
import energy_balance
import pytest

@pytest.fixture
def setup():
    energy_balance.init()
```

Define a fixture

tests/test_energy_balance.py

```
import energy_balance
from testing import setup

def test_energy_in_albedo_max(setup):
    assert energy_balance.energy_in(1) == 0
```

Use it in tests that need it

Negative tests

“We expect this to fail with an exception”

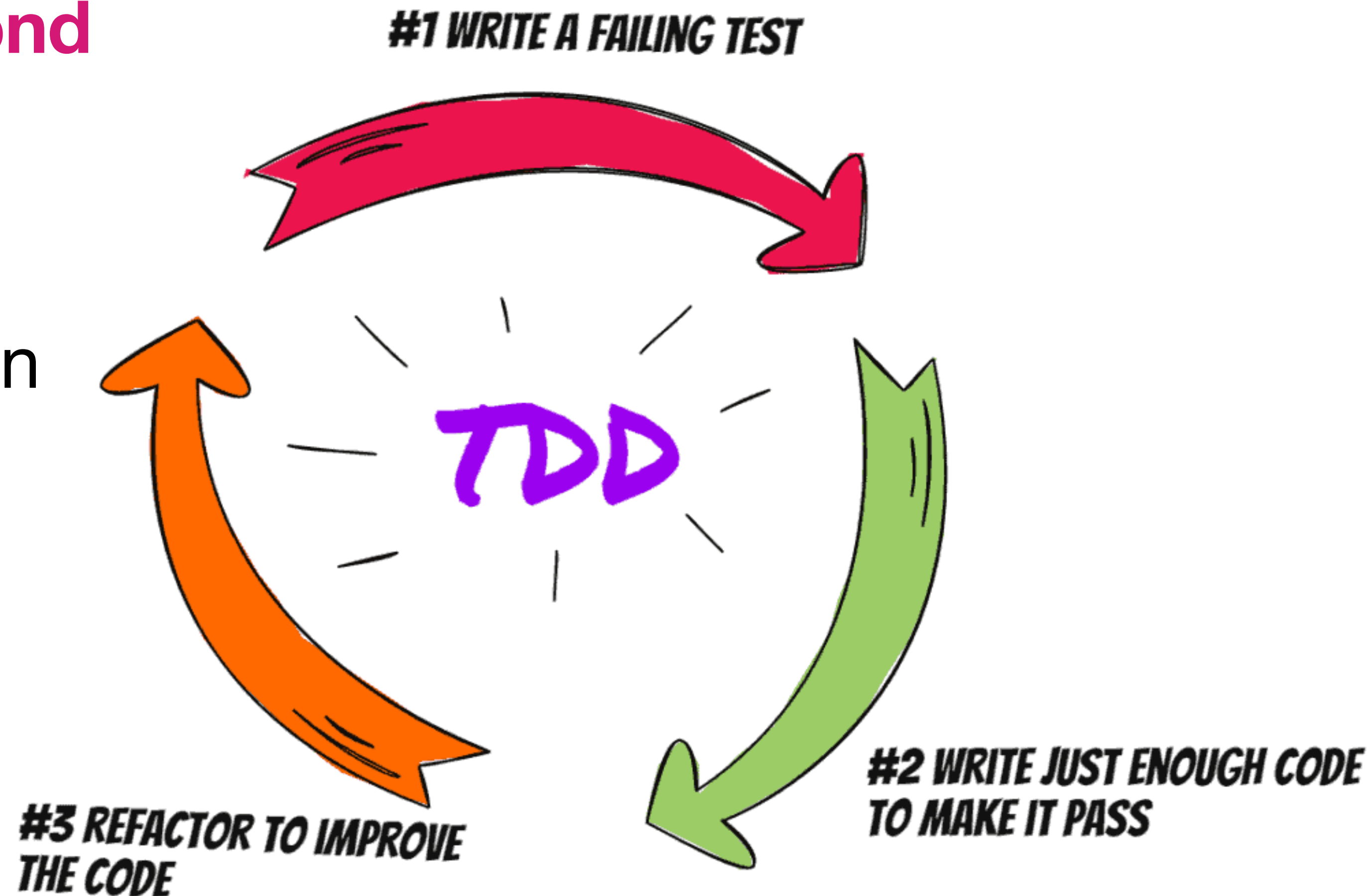
```
def test_energy_in_invalid(setup):  
    albedo = -0.1  
    with pytest.raises(ValueError):  
        energy_balance.energy_in(albedo)
```

Unit test basics (3)

- **General principle:** try to cover a representative sample of input space.
- For data structures:
 - Empty data structure
 - Small data structures
 - All constructor shapes (i.e., all small sizes of binary tree)
 - Very large?

Test-Driven Development (TDD)

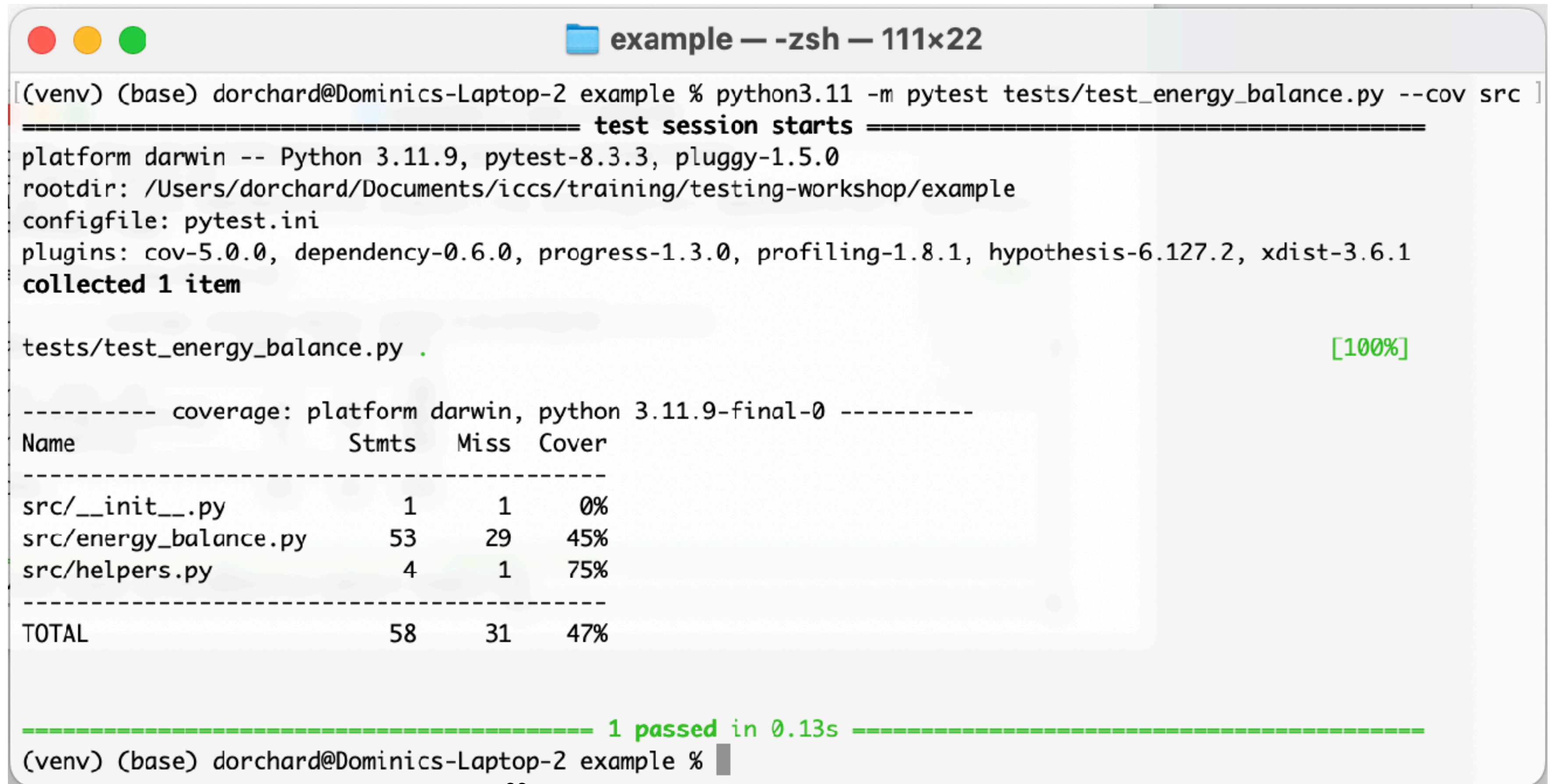
- **Specify first, implement second**
- TDD cycle is quite fine grained
 - #1 could be repeated first
- Encourage more modular design
- ‘Correctness first’ mindset



Coverage

What % of *control-flow paths* have a test?

```
pytest tests --cov path
```



```
example — -zsh — 111x22
[venv] (base) dorchard@Dominics-Laptop-2 example % python3.11 -m pytest tests/test_energy_balance.py --cov src ]
===== test session starts =====
platform darwin -- Python 3.11.9, pytest-8.3.3, pluggy-1.5.0
rootdir: /Users/dorchard/Documents/iccs/training/testing-workshop/example
configfile: pytest.ini
plugins: cov-5.0.0, dependency-0.6.0, progress-1.3.0, profiling-1.8.1, hypothesis-6.127.2, xdist-3.6.1
collected 1 item

tests/test_energy_balance.py . [100%]

----- coverage: platform darwin, python 3.11.9-final-0 -----
Name                               Stmts  Miss  Cover
-----
src/__init__.py                      1      1    0%
src/energy_balance.py               53     29   45%
src/helpers.py                      4      1   75%
-----
TOTAL                               58     31   47%

===== 1 passed in 0.13s =====
[venv] (base) dorchard@Dominics-Laptop-2 example %
```

Coverage

What % of *control-flow paths* have a test?

```
pytest tests --cov path
```

To find out which control-flow paths (per line) are not covered:

```
pytest tests/ --cov src --cov-report term-missing
```


Part II - Integration tests

“End-to-end”

Part III - Property-based tests

Does integration + unit testing scale?

```
def test_square():  
    assert square(0) == 0  
    assert square(1) == 1  
    assert square(4) == 16  
    assert square(-1) == 1  
    assert isnan(square(nan))
```

A small subset of the input space

Thinking of corner cases is hard
Writing lots of cases is time consuming

Can we automate?

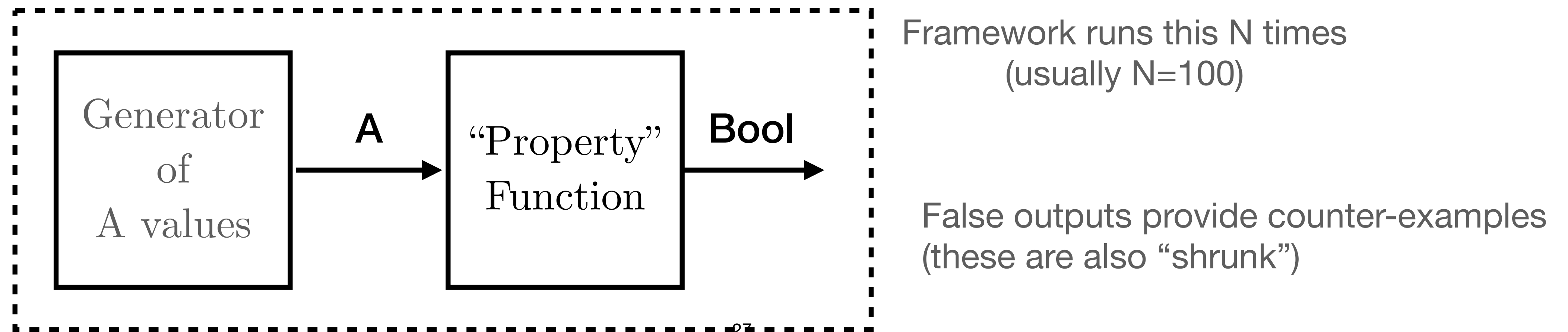
Property-based testing basis

- Tests based on a general property over broad input space

```
def property(x : str) -> bool:  
    return (reverse(reverse(x)) == x)
```

“Generator” for `str` systematically specialises / randomises inputs, finding counterexamples

see QuickCheck (Haskell), hypothesis (Python)

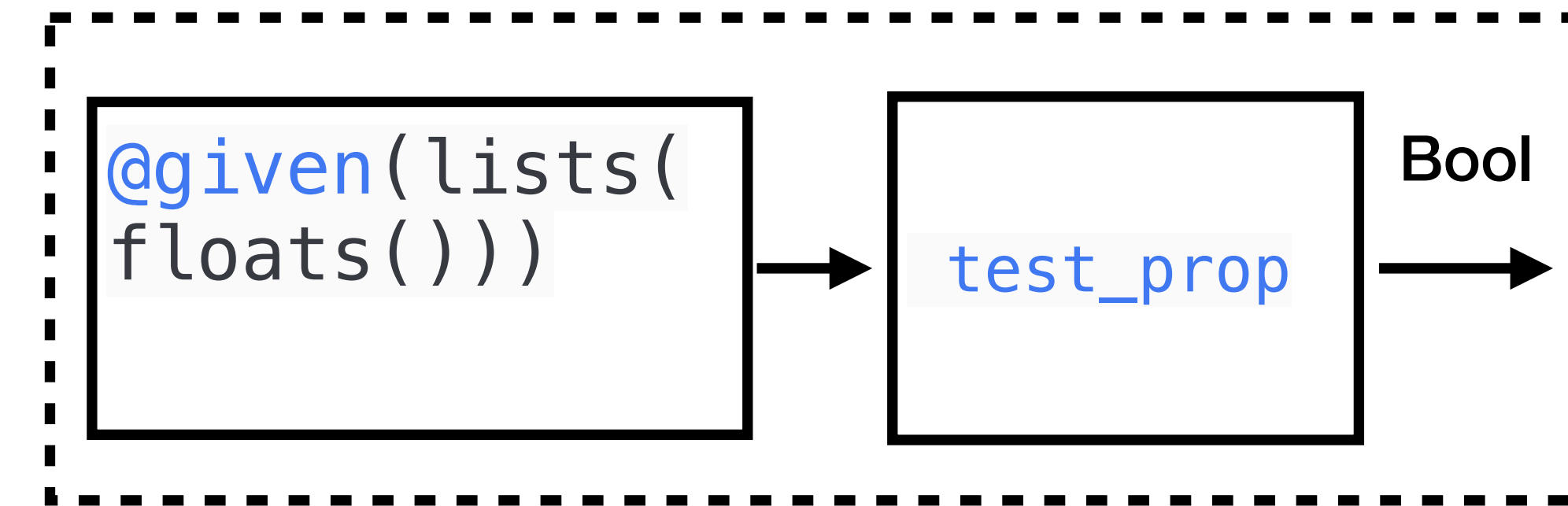


Python hypothesis library; strategies

- Generators called “strategies”

```
from hypothesis import given
from hypothesis.strategies import floats, lists

@given(lists(floats()))
def test_prop(input_data):
    ...
```



- Automatically generates 100 inputs of the given form; runs `test_prop` with each
- Strategies can be configured to choose a subset of the domain, e.g.,

```
floats(allow_infinity=False, allow_nan=False)
```

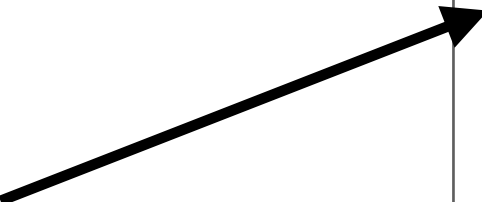
Custom / composite strategies

Example: two random lists of the same length

- Define a function, expecting a function as input which 'draws out' values from other strategies (**draw**)

Defines a strategy that is built out of potentially arbitrarily many other strategies.

@composite provides a callable draw to dynamically draw a value from any strategy



```
@composite
# Generate two lists of the same length
def same_len_float_lists(draw):
    list1 = draw(lists(floats()))
    list2 = draw(lists(floats(),
                        min_size=len(list1),
                        max_size=len(list1)
                    ))
    return (list1, list2)

@given(same_len_float_lists())
def test_msgerror2(input_data):
    model_data, obs_data = input_data
    ...
```


Diverse Inputs + Strong Properties $\implies$ Strong(er) Tests

How to create strong properties?

Defining useful properties

“Avoid replicating your code in your tests.” [1]

1. Postconditions - Relation between return values/arguments; what is true after?
e.g., for reverse/sort, every element in input list is also in output (and vice versa)
[but on its own this is not sufficient]
2. Validity Testing - Every operation should return valid results
Define a notion of validity as a boolean-valued function
3. Algebraic properties - Involution? Commutativity? Associativity? Unitality?
4. Metamorphic properties - Related calls return related results
Relevant in science: invariance / symmetries
Conservation of energy/momentum/mass etc.

Final thoughts

- Powerful technique
- Combine with other approaches
- Custom generators+shrinkers
(further work)
- Frameworks exist in most languages

Property Testing for Ocean Models. Can We Specify It?

Deepak A. Cherian
Golden, Colorado
deepak@cherian.net

I take inspiration from the property-testing literature, particularly the work of Prof. John Hughes [17], and explore how such ideas might be applied to numerical models of the ocean. Specifically, I ask whether geophysical fluid dynamics (GFD) theory, expressed as property tests, might be used to address the oracle problem of testing the correctness of ocean models. I propose that a number of simple idealized GFD problems can be framed as property tests. These examples clearly illustrate how physics naturally lends itself to specifying property tests. Which of these proposed tests might be most feasible and useful, remains to be seen.

1 Introduction

1.1 Background

Simulated possible projections of the Earth's climate by Earth System Models are an important tool in understanding, adapting to, and mitigating the effects of climate change. Such models bring together a number of individual component models (e.g. the atmosphere, ocean, land, rivers, and more) coupled to each other through modeled interactions at the interfaces between these components (for example, the sea surface). The complexity of each individual component is vast, the complexity of a full Earth System Model is staggering (Figure 1).

Consider the ocean. The ocean is a thin shell of fluid on the Earth¹. The ocean is “stratified”, meaning the density of seawater varies with depth so that lighter fluid overlays denser fluid. Commonly, warm and fresh (low salinity) water overlays colder, saltier water. The equations that describe the motion of the ocean are the Navier-Stokes equations specialized for rotating fluids. These equations express the

¹compare its depth of ~5 km to the radius of the earth (~6400 km)

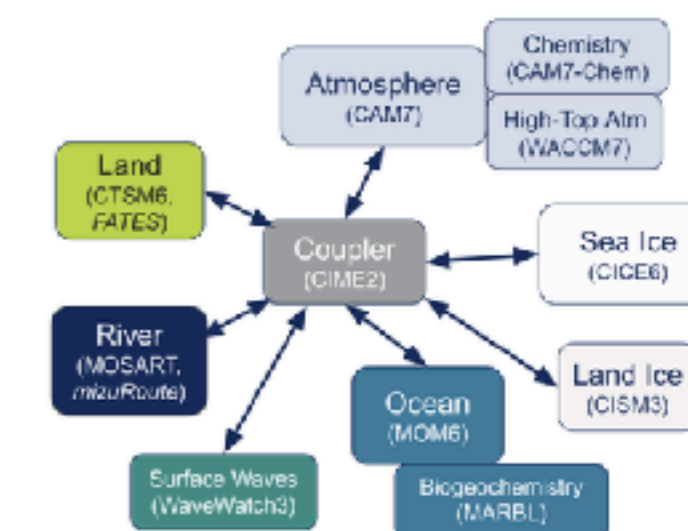


Figure 1: Schematic of the Community Earth System Model version 3 (CESM3) [20].

Thanks- and happy testing!



<https://iccs.cam.ac.uk>